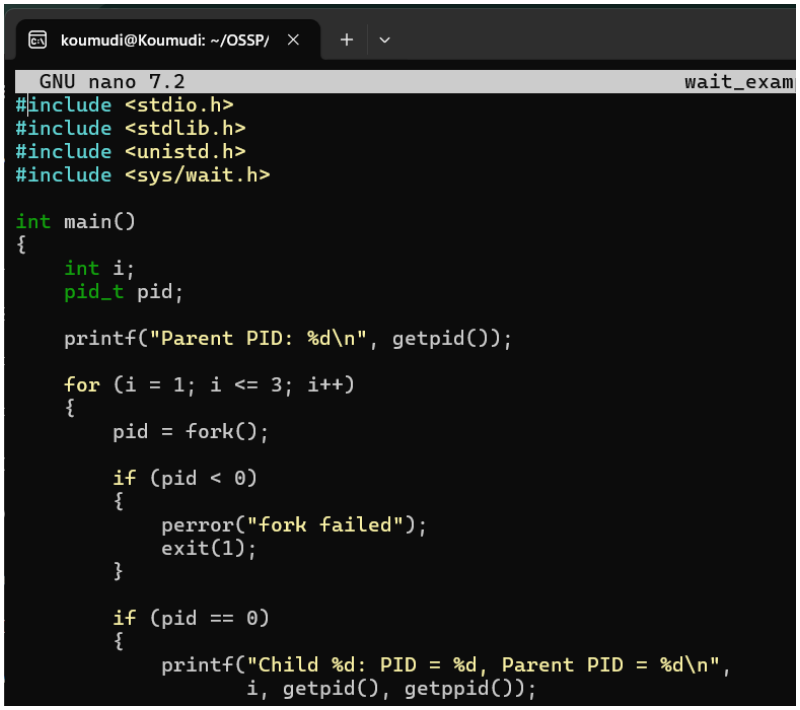
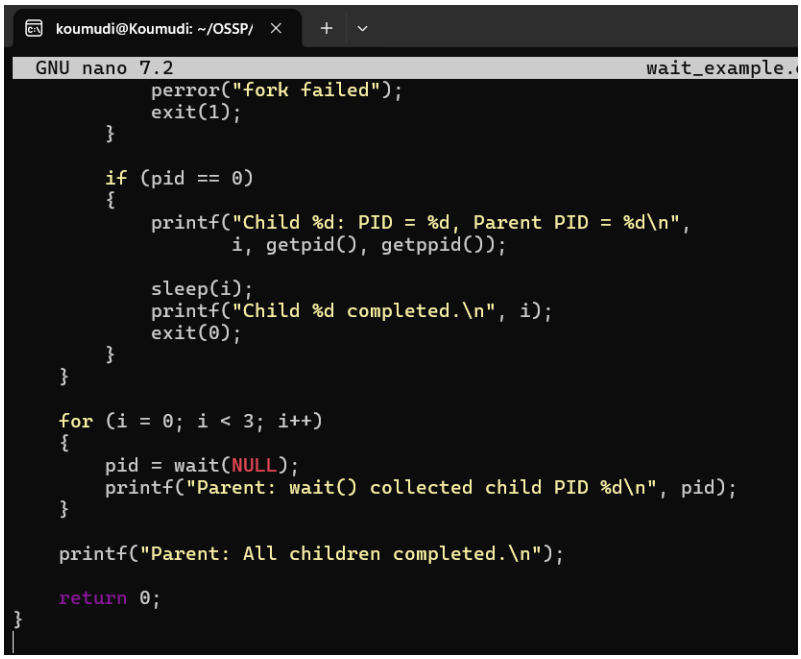
PRACTICAL (OSSP)

SESSION 4

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.



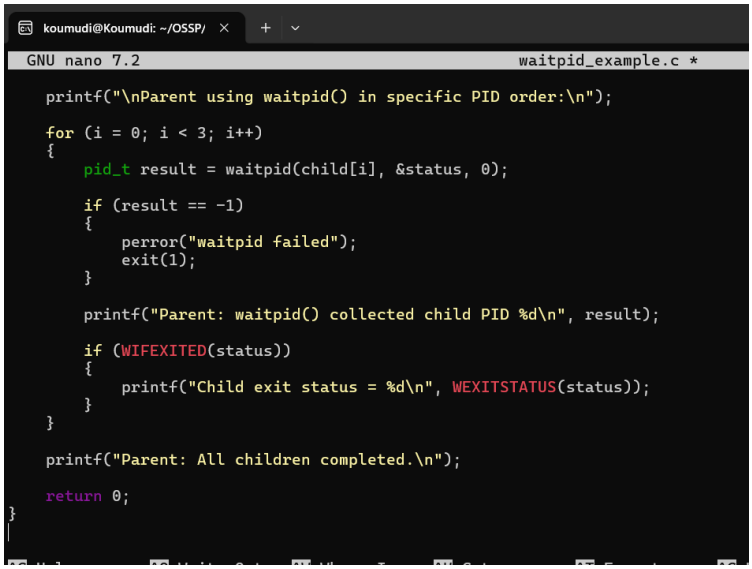


```
koumudi@Koumudi:~$ cd ~/OSSP/PRACTICAL
mkdir -p Session_4
cd Session_4
pwd
/home/koumudi/OSSP/PRACTICAL/Session_4
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ nano wait_example.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ gcc wait_example.c -o wait_example
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ ./wait_example
Parent PID: 595
Child 1: PID = 596, Parent PID = 595
Child 2: PID = 597, Parent PID = 595
Child 3: PID = 598, Parent PID = 595
Child 1 completed.
Parent: wait() collected child PID 596
Child 2 completed.
Parent: wait() collected child PID 597
Child 3 completed.
Parent: wait() collected child PID 598
Parent: All children completed.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$
```

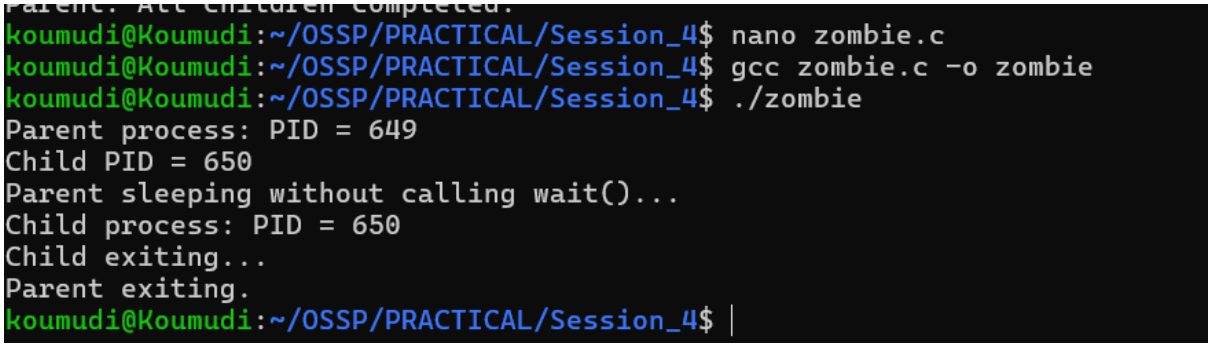
```
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ nano waitpid_example.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ gcc waitpid_example.c -o waitpid_example
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ ./waitpid_example
Parent PID: 623
Child 1: PID = 624, Parent PID = 623
Child 2: PID = 625, Parent PID = 623

Parent using waitpid() in specific PID order:
Child 3: PID = 626, Parent PID = 623
Child 3 completed.
Child 2 completed.
Child 1 completed.
Parent: waitpid() collected child PID 624
Child exit status = 10
Parent: waitpid() collected child PID 625
Child exit status = 11
Parent: waitpid() collected child PID 626
Child exit status = 12
Parent: All children completed.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$
```

koumudi@Koumudi: ~/OSSP/

```
  GNU nano 7.2                                          waitpid_exam
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child[3];
    int i;
    int status;

    printf("Parent PID: %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
```



TASK 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.



koumudi@Koumudi: ~/OSSP/

GNU nano 7.2 zombie

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping without calling wait()...\n");
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2 zombie

```c
    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping without calling wait()...\n");

        sleep(20);

        printf("Parent exiting.\n");
    }

    return 0;
}
```

koumudi@Koumudi: ~

```
koumudi@Koumudi:~$ ps -el | grep zombie
0 S  1000     727     381  0  80   0 -   672 hrtime pts/0    00:00:00 zombie
1 Z  1000     728     727  0  80   0 -     0 -      pts/0    00:00:00 zombie
koumudi@Koumudi:~$
```

```
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ nano zombie.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ gcc zombie.c -o zombie
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$ ./zombie
Parent process: PID = 747
Child PID = 748
Parent waiting for child using wait()...
Child process: PID = 748
Child exiting...
Child process collected. No zombie remains.
Parent exiting.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_4$
```